

HW3答案优化

(20) 1、写出 $(4365)_8-(3412)_8$ 的计算过程，两数都是无符号八进制数，结果也必须使用八进制表示。

```
  4 3 6 5
- 3 4 1 2
-----
```

$(753)_8$

(20) 2、已知： $X=-13/16$ ， $Y=7/16$ ，用Booth算法求 $X \times Y=?$ 请写出过程。结果用补码表示。

按照ppt上保留 小数点 步骤也可。

转换为二进制补码定点小数（设字长5位：1符号位 + 4小数位）

- X被乘数，Y乘数
- $X=-0.8125=-0.1101$ （原码 1.1101）→ 补码：1.0011
- $Y=4375=0.0111$ （正数，补码同）→ 0.0111
为了便于运算，将小数视为整数乘以 2^4 （即扩大16倍），则整数表示：
- $X'=-13$ ，原码 11101，补码 10011
- $Y'=7$ ，5位补码 00111

Booth 算法运算过程（整数补码）

- 设 部分积 $A=X'$ 补码 = 10011
- 设 $S=-X'$ 补码 = 01101（对 10011 取反加1得 01101）
- 设 P 初始为 0（高位部分，5位），乘数 $Q=Y'$ = 00111（低位部分，5位）
- 附加位(辅助位) $Q_{-1}=0$
循环次数 $n=5$ （乘数位数）。
每一步检查 Q 的最低位 (Q_0, Q_{-1})，决定操作(01+X,10-X)：

循环	Q_0, Q_{-1}	操作	P（高位）	Q（低...	Q_{-1}
初态	-	-	00000	00111	0

循环	Q ₀ , Q ₋₁	操作	P（高位）	Q（低...	Q ₋₁
1	(1,0)	$P \leftarrow P - X' = P + S$	$00000 + 01101 = 01101$	00111	-
-	-	高位右移，低位移入高位末尾	00110	10011	1
2	(1,1)	算术右移	00011	01001	1
3	(1,1)	算术右移	00001	10100	1
4	(0,1)	$P \leftarrow P + X' = P + A$	$00001 + 10011 = 10100$	10100	-
-	-	右移	11010	01010	0
5	(0,0)	算术右移	11101	00101	0

最终 P 高5位 = 11101，Q 低5位 = 00101，合并为10位结果： 1110100101。

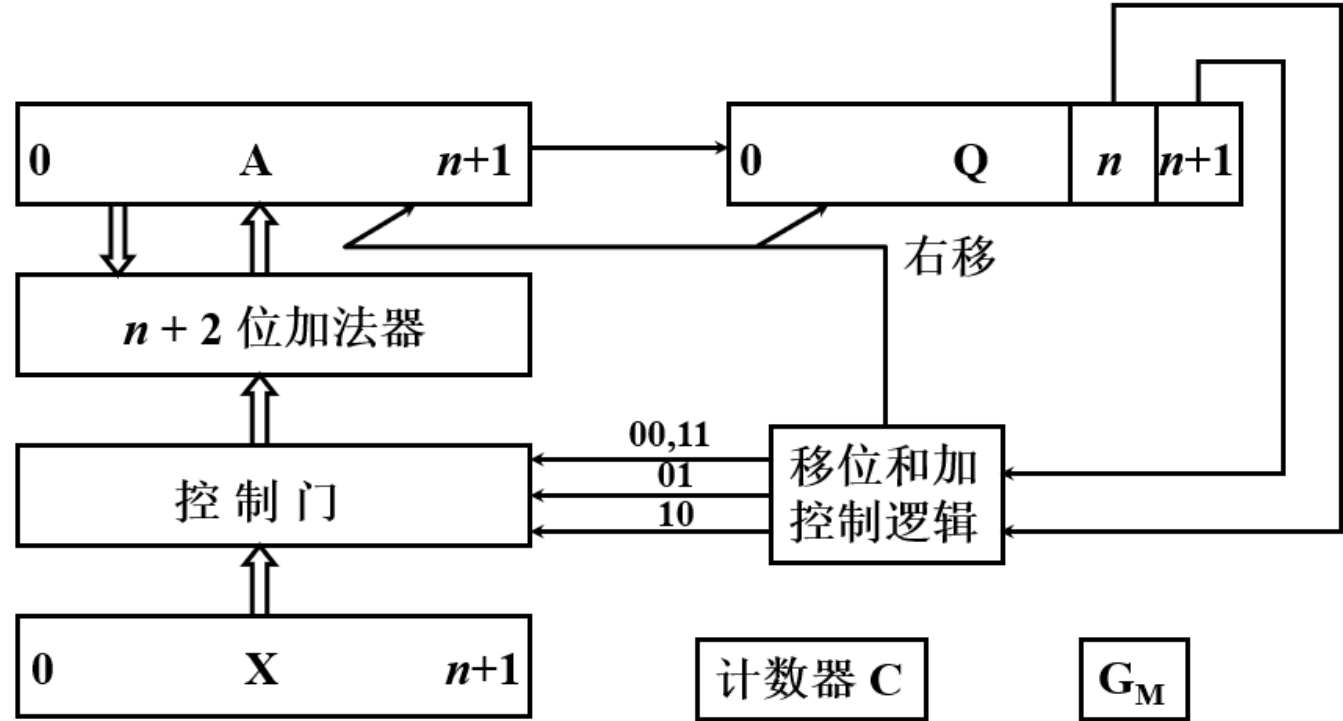
将乘积结果还原为小数

乘积的补码整数 = 1110100101（10位），因为之前将原数扩大了 $16 \times 16 = 2^8$ 倍，故实际为 11.10100101

$$(X \times Y)_{\text{补}} = 1.10100101_2$$

推荐学习

Booth 算法的硬件实现



A、X、Q 均 $n+2$ 位
移位和加受末两位乘数控制

<https://blog.csdn.net/qq1350975694>

Booth算法（Booth Multiplication Algorithm）是一种：用补码直接进行有符号数乘法的算法。

它的核心目的：

- 正确处理正负数
- 减少连续“1”带来的重复加法
- 适用于计算机硬件实现

Booth算法不直接看某一位 Y_i ，而是看 (Y_i, Y_{i-1}) 两位组合。

规则如下：

$Y_i Y_{i-1}$	操作
00	不操作
11	不操作
01	+X
10	-X

随后，对整个寄存器进行“算术右移”。

② 为何如此？

例如：0111100，普通乘法需要 $+X +X +X +X$ 。但Booth发现，0111100 其实等价于 $1000000 - 0000100$ ，即 $+X \ll 6, -X \ll 2$ 。从而减少运算次数。

十进制

$$9999 \times 9990 \\ = (10000 - 1) \times 9990$$

$-5 \times -1 = 5$
 $1011 \times 1111 = 10100101$
加法进行了3次
↓
如果是？
 -1×-2 需要进行几次加法？
 $1111 \times 1110 =$ 也是3次
能不能减少加法的次数呢？
【就可以减少一些时钟周期】

二进制

$$1111 \times 1110 \\ = (10000 - 1) \times 1110 \\ \text{only 2}$$

(60) 3、用课本图3-11中的硬件结构计算 $(74)_8$ 除以 $(21)_8$ ，并给出一个类似于图3-10中的表。要求给出每一步中各个寄存器的值。其中被除数和除数都是八进制无符号整数。

题目图片

迭代次数	步骤	商	除数	余数
0	初始值	0000	0010 0000	0000 0111
1	1: 余数=余数-除数	0000	0010 0000	①110 0111
	2b: 余数<0⇒+除数, 商左移, 商最低位=0	0000	0010 0000	0000 0111
	3: 除数右移	0000	0001 0000	0000 0111
2	1: 余数=余数-除数	0000	0001 0000	①111 0111
	2b: 余数<0⇒+除数, 商左移, 商最低位=0	0000	0001 0000	0000 0111
	3: 除数右移	0000	0000 1000	0000 0111
3	1: 余数=余数-除数	0000	0000 1000	①111 1111
	2b: 余数<0⇒+除数, 商左移, 商最低位=0	0000	0000 1000	0000 0111
	3: 除数右移	0000	0000 0100	0000 0111
4	1: 余数=余数-除数	0000	0000 0100	①000 0011
	2a: 余数>=0⇒商左移, 商最低位=1	0001	0000 0100	0000 0011
	3: 除数右移	0001	0000 0010	0000 0011
5	1: 余数=余数-除数	0001	0000 0010	①000 0001
	2a: 余数>=0⇒商左移, 商最低位=1	0011	0000 0010	0000 0001
	3: 除数右移	0011	0000 0001	0000 0001

图 3-10 除法的例子，采用图 3-9 中的算法。图中圈起来的位用于决定下一步的操作

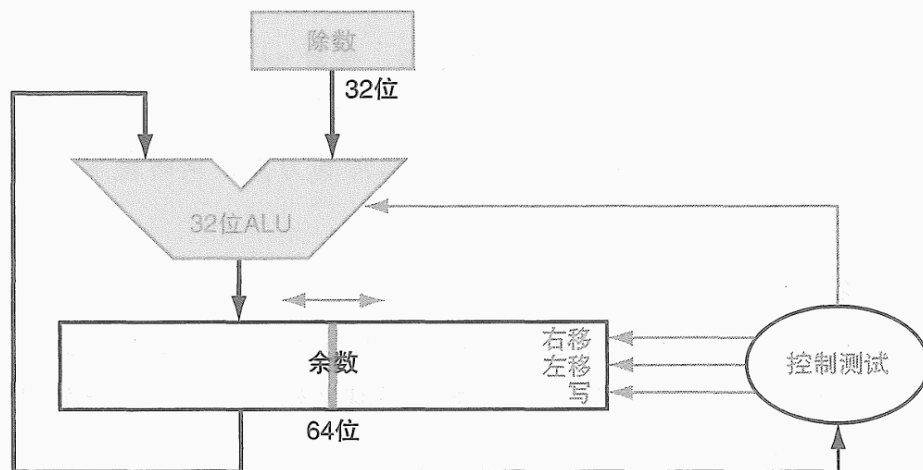


图 3-11 除法器的一种改进版本。除数寄存器、ALU、商寄存器都是 32 位，只有余数寄存器为 64 位。与图 3-8 相比，ALU 和除数寄存器都是位宽减半，余数进行左移。这个结构将商寄存器和余数寄存器的右半部分进行了拼接（正如图 3-6 中的那样，余数寄存器应该是 65 位以保证加法器的进位不会丢失）

3.19 In these solutions a 1 or a 0 was added to the Quotient if the remainder was greater than or equal to 0. However, an equally valid solution is to shift in a 1 or 0, but if you do this you must do a compensating right shift of the remainder (only the remainder, not the entire remainder/quotient combination) after the last step.

$$74/21 = 3 \text{ remainder } 11$$

Step	Action	Divisor	Remainder/Quotient
0	Initial Vals	010 001	000 000 111 100
1	R<<	010 001	000 001 111 000
	Rem=Rem-Div	010 001	111 000 111 000
	Rem<0, R+D	010 001	000 001 111 000
2	R<<	010 001	000 011 110 000
	Rem=Rem-Div	010 001	110 010 110 000
	Rem<0, R+D	010 001	000 011 110 000
3	R<<	010 001	000 111 100 000
	Rem=Rem-Div	010 001	110 110 110 000
	Rem<0, R+D	010 001	000 111 100 000
4	R<<	010 001	001 111 000 000
	Rem=Rem-Div	010 001	111 110 000 000
	Rem<0, R+D	010 001	001 111 000 000

Step	Action	Divisor	Remainder/Quotient
5	R<<	010 001	011 110 000 000
	Rem=Rem-Div	010 001	111 110 000 000
	Rem>0, R0=1	010 001	001 101 000 001
6	R<<	010 001	011 010 000 010
	Rem=Rem-Div	010 001	001 001 000 010
	Rem>0, R0=1	010 001	001 001 000 011

$$74_8 = 111\ 100_2; D = 21_8 = 010\ 001_2$$

(-D_补 = 101 111, 因为无符号, 所以自己计算可以用补码, 但是需要注意正负)

迭代次数n=商的位数

恢复余数除法

步	左移后RQ	R-D	是否恢复	商位
初值	000000 111100	-	-	-
1	000001 111000	110000(负)	恢复→000001	0
2	000011 110000	110010(负)	恢复→000011	0
3	000111 100000	110110(负)	恢复→000111	0
4	001111 000000	111110(负)	恢复→001111	0
5	011110 000000	001101(正)	不恢复	1
	001101 000001			

步	左移后RQ	R-D	是否恢复	商位
6	011010 000010	001001(正)	不恢复	1
	001001 000011			

不恢复余数除法

步骤	左移后RQ	操作	运算结果R	商位
初值	000000 111100	-	-	-
1	000001 111000	R-D	110000(负)	0
2	100001 110000	R+D	110010(负)	0
3	100101 100000	R+D	110110(负)	0
4	101101 000000	R+D	111110(负)	0
5	111100 000000	R+D	001101(正)	1
6	011010 000010	R-D	001001(正)	1
	001001 000011			

架构对比

图 3-8 给出了模拟小学除法算法的硬件结构。在开始时，32 位的商寄存器设为 0。算法每次迭代将除数向右移一位，因此开始时需要将除数放置在 64 位除数寄存器的左半边，然后每次右移一位来与被除数对齐。余数寄存器初始化为被除数。

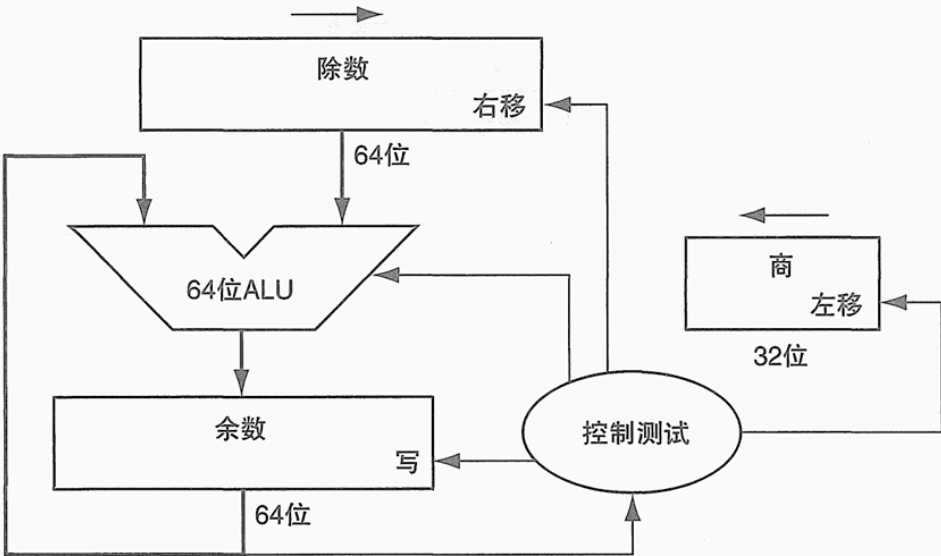


图 3-8 第一版除法器硬件结构。除数寄存器、ALU、余数寄存器都是 64 位宽，只有商寄存器是 32 位宽。32 位的除数开始放置在除数寄存器的左半部分，然后每次迭代右移一位。余数寄存器初始化为被除数。控制逻辑决定何时对除数和商寄存器进行移位，以及何时将新值写入余数寄存器

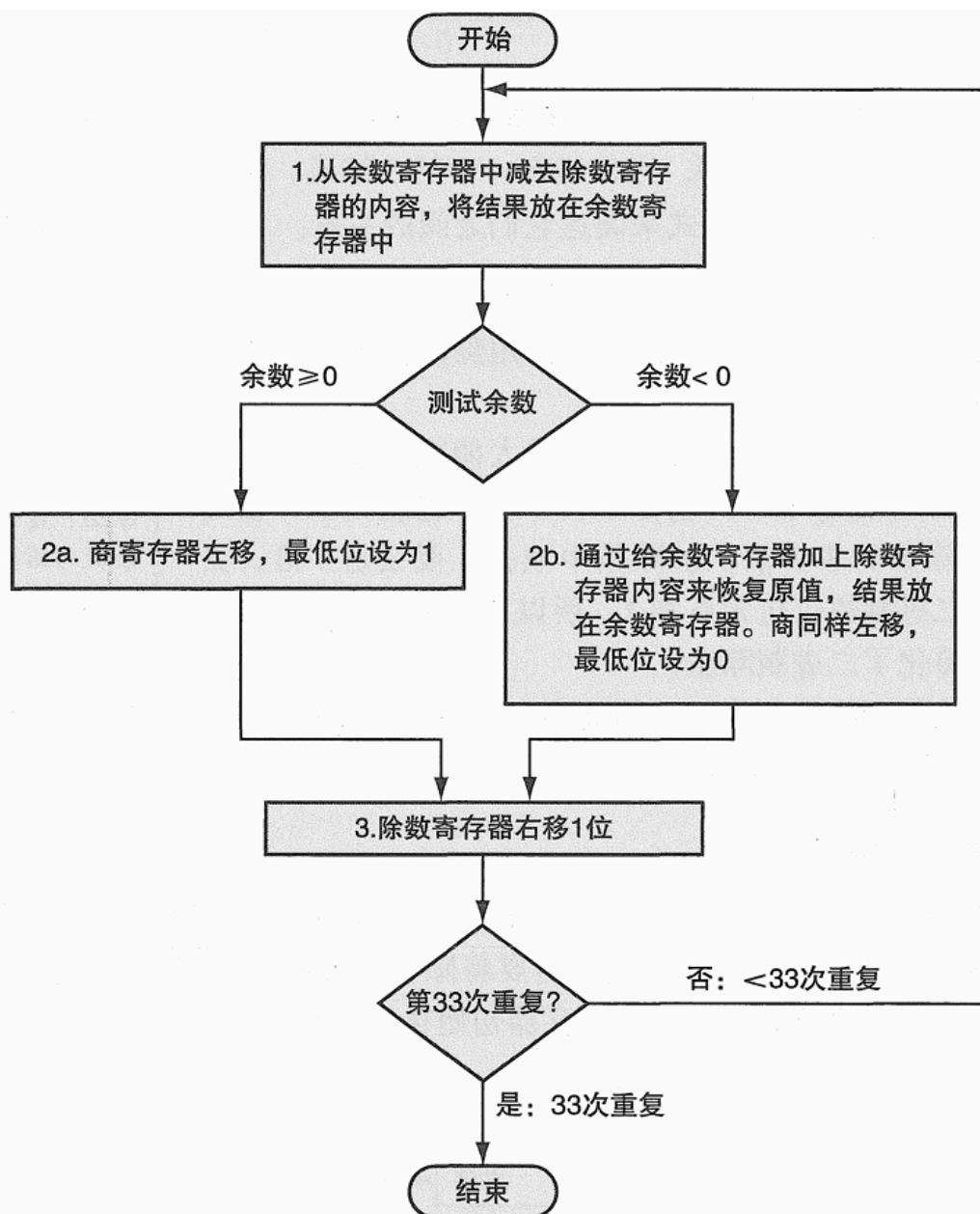


图 3-9 使用图 3-8 中硬件的除法算法。如果余数不为负，则将除数从被除数中减去，然后在第 2a 步取商为 1。如果第 1 步之后余数为负，则意味着除数不能从被除数中减去，所以在第 2b 步中取商为 0 并将除数加到余数上，即做第 1 步减法的逆操作。在第 3 步，进行最后的移位，根据下一个迭代的被除数，将除数适当对齐。这些步骤将要重复 33 次

图 3-8 给出了模拟小学除法算法的硬件结构。在开始时，32 位的商寄存器设为 0。算法每次迭代将除数向右移一位，因此开始时需要将除数放置在 64 位除数寄存器的左半边，然后每次右移一位来与被除数对齐。余数寄存器初始化为被除数。

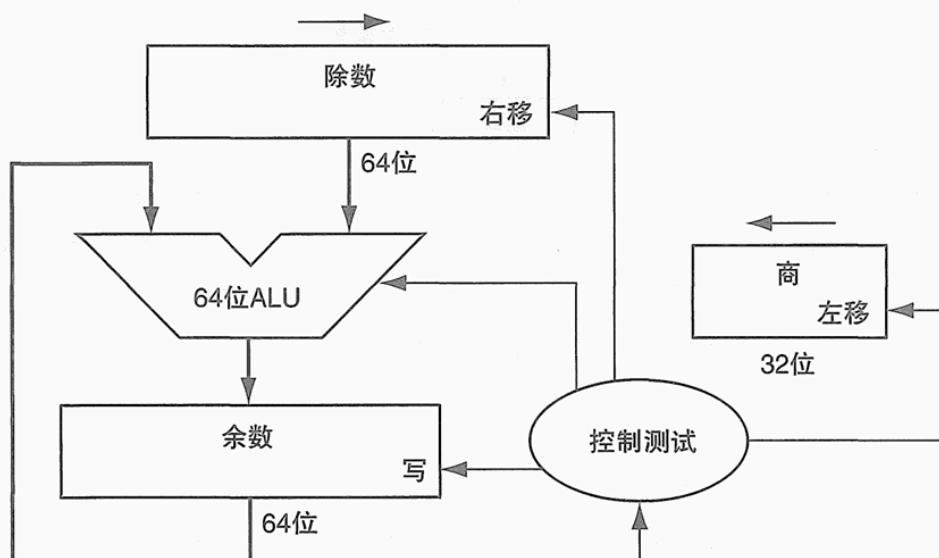
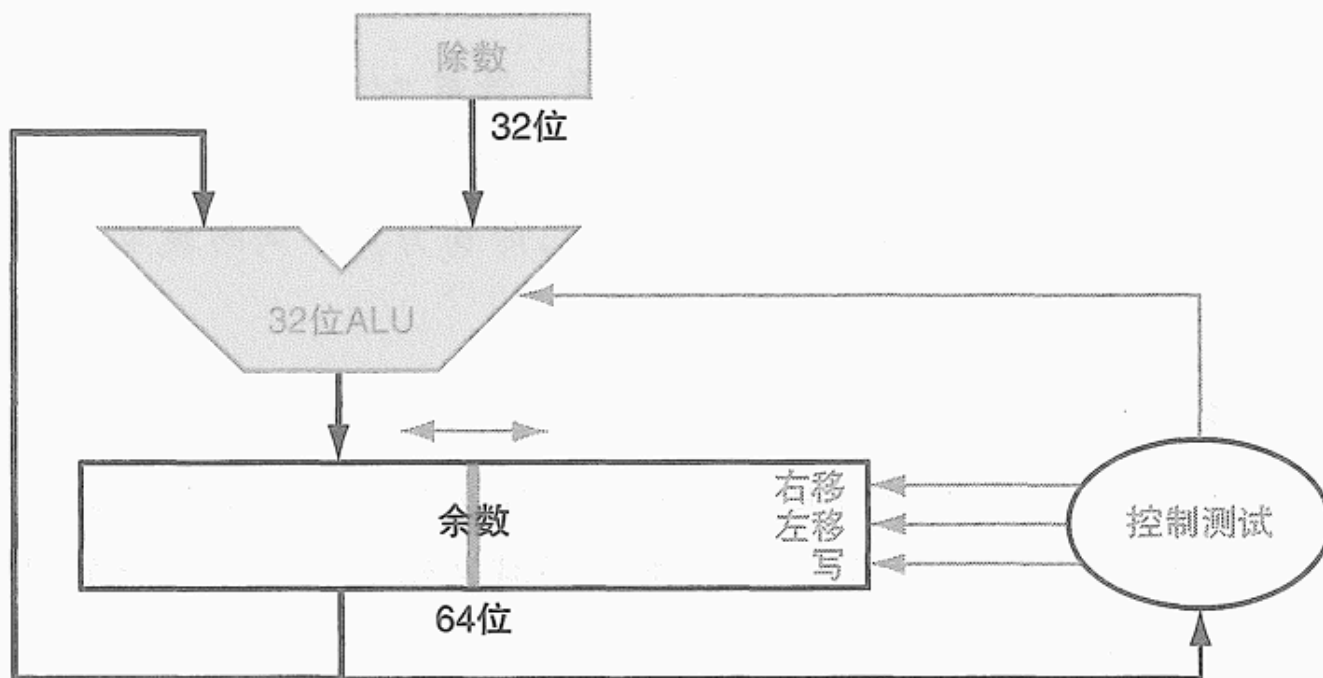
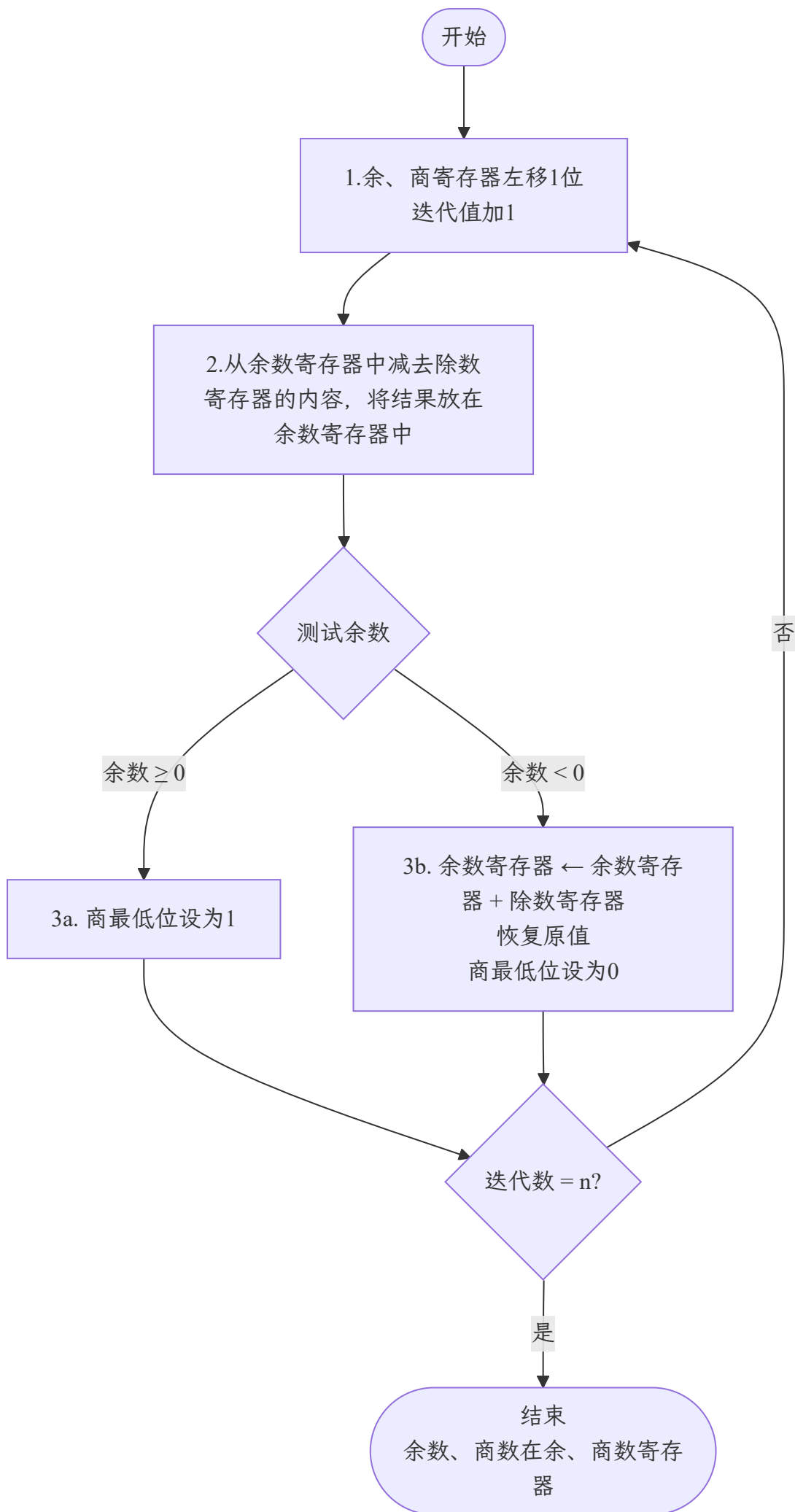


图 3-8 第一版除法器硬件结构。除数寄存器、ALU、余数寄存器都是 64 位宽，只有商寄存器是 32 位宽。32 位的除数开始放置在除数寄存器的左半部分，然后每次迭代右移一位。余数寄存器初始化为被除数。控制逻辑决定何时对除数和商寄存器进行移位，以及何时将新值写入余数寄存器





[恢复余数除法](#) | [CodeBrick 408 计算机组成原理](#)
[计算机组成_中国大学MOOC\(慕课\)](#)